DESAFIO DE DEV - MATHEUS VASCONCELOS

O que foi feito?

As rotas implementadas foram as seguintes:

- routes.post("/plantas", PlantController.createPlant);
Responsável por criar uma nova planta e armazená-la no dataset.

- routes.get("/plantas", PlantController.readAllPlants);
Usado para retornar todas as plantas armazenadas, com possível utilização de filtros opcionais para as variáveis de luminosidade e/ou petFriendly.

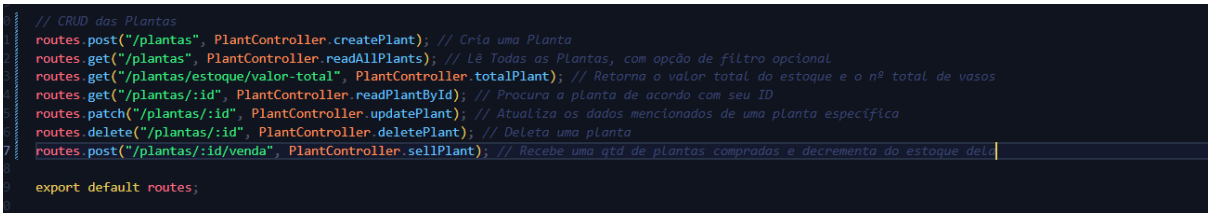
- routes.get("/plantas/estoque/valor-total", PlantController.totalPlant);
Retorna um objeto com o valor total da loja (somatório de preço * estoque de todas as plantas) e com o total de vasos atualmente na loja.

- routes.get("/plantas/:id", PlantController.readPlantById);
Retorna uma planta, procurada através de seu id único.

- routes.patch("/plantas/:id", PlantController.updatePlant);
Atualiza os dados escolhidos de uma planta já existente.

- routes.delete("/plantas/:id", PlantController.deletePlant);
Deleta uma planta específica do dataset.

- routes.post("/plantas/:id/venda", PlantController.sellPlant);
Recebe uma quantidade: { “quantidade”: qtd } que será decrementada de uma das plantas, com impedimento caso o número seja maior do que o estoque atual (ou seja, o estoque ficaria menor do que 0) ou caso seja um número <= 0 (que não representa uma venda válida).



```
// CRUD das Plantas
routes.post("/plantas", PlantController.createPlant); // Cria uma Planta
routes.get("/plantas", PlantController.readAllPlants); // Lê Todas as Plantas, com opção de filtro opcional
routes.get("/plantas/estoque/valor-total", PlantController.totalPlant); // Retorna o valor total do estoque e o nº total de vasos
routes.get("/plantas/:id", PlantController.readPlantById); // Procura a planta de acordo com seu ID
routes.patch("/plantas/:id", PlantController.updatePlant); // Atualiza os dados mencionados de uma planta específica
routes.delete("/plantas/:id", PlantController.deletePlant); // Deleta uma planta
routes.post("/plantas/:id/venda", PlantController.sellPlant); // Recebe uma qtd de plantas compradas e decrementa do estoque dela

export default routes;
```

Nenhum bônus foi implementado no desafio.
REGRAS DE NEGÓCIO:

1. Todos os campos são obrigatórios no cadastro.
2. preco sempre maior que zero; regaACadaDias pelo menos 1.

3. estoque não pode ser negativo, mas pode ser zero (planta esgotada).
4. luminosidade só aceita os três valores do enum.
5. Dados inválidos (no cadastro, na edição, nos filtros ou na venda) retornam 400 com uma mensagem
clara.
6. Um id que não existe retorna 404.
7. A venda nunca deixa o estoque negativo: sem estoque suficiente, retorna 400 e não altera nada.

Como foi feito?

Criação de novas Plantas

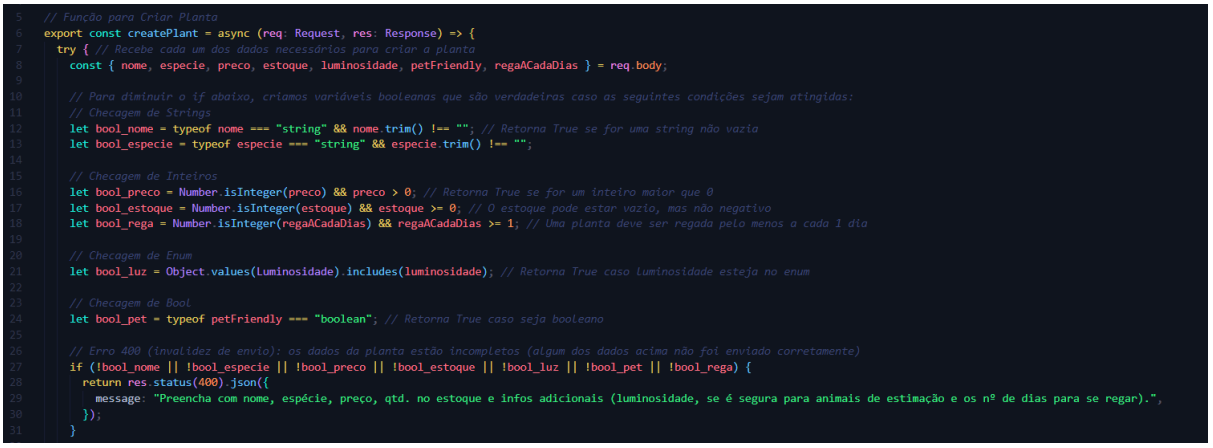
No arquivo schema.prisma, criamos um novo modelo chamado “Plant”, que armazena todas as informações necessárias e solicitadas pelo problema (id, nome, espécie, preço da planta, quantidade no estoque, luminosidade necessária, segurança com pets, nº de vezes para regar e as variáveis de criação e atualização do modelo). Na espécie, o atributo “@unique” foi usado porque **plantas diferentes possuem nomes científicos distintos**; dessa forma, garantimos que, mesmo que uma planta possa ter o mesmo nome que outra, haverá distinção entre elas através do nome científico, único e exclusivo da espécie procurada.



No arquivo de repositório (PlantRepository.ts), a função não sofreu alterações comparado ao exemplo dado durante o treinamento, dado que a funcionalidade é exatamente a mesma.

```ts
// Cria uma nova planta
export const createPlant = async (data: Prisma.PlantCreateInput) => {
    return await prisma.plant.create({
        data: data,
    });
};
```

Entretanto, no arquivo de controle (PlantController.ts), incluímos um método de checagem para cada uma das variáveis solicitadas na adição de uma nova planta ao estoque, de forma que, caso o objeto de entrada não contenha um dos elementos necessário (ou então apresente um termo inválido) a entrada é negada e o item não é adicionado.



```ts
// Função para Criar Planta
export const createPlant = async (req: Request, res: Response) => {
  try { // Recebe cada um dos dados necessários para criar a planta
    const { nome, especie, preco, estoque, luminosidade, petFriendly, regaACadaDias } = req.body;

    // Para diminuir o if abaixo, criamos variáveis booleanas que são verdadeiras caso as seguintes condições sejam atingidas:
    // Checagem de Strings
    let bool_nome = typeof nome === "string" && nome.trim() !== ""; // Retorna True se for uma string não vazia
    let bool_especie = typeof especie === "string" && especie.trim() !== "";

    // Checagem de Inteiros
    let bool_preco = Number.isInteger(preco) && preco > 0; // Retorna True se for um inteiro maior que 0
    let bool_estoque = Number.isInteger(estoque) && estoque >= 0; // O estoque pode estar vazio, mas não negativo
    let bool_rega = Number.isInteger(regaACadaDias) && regaACadaDias >= 1; // Uma planta deve ser regada pelo menos a cada 1 dia

    // Checagem de Enum
    let bool_luz = Object.values(Luminosidade).includes(luminosidade); // Retorna True caso luminosidade esteja no enum

    // Checagem de Bool
    let bool_pet = typeof petFriendly === "boolean"; // Retorna True caso seja booleano

    // Erro 400 (invalidez de envio): os dados da planta estão incompletos (algum dos dados acima não foi enviado corretamente)
    if (!bool_nome || !bool_especie || !bool_preco || !bool_estoque || !bool_luz || !bool_pet || !bool_rega) {
      return res.status(400).json({
        message: "Preencha com nome, espécie, preço, qtd. no estoque e infos adicionais (luminosidade, se é segura para animais de estimação e os nº de dias para se regar).",
      });
    }
```

Através dessa checagem, buscamos barrar as entradas de acordo com o tipo de cada uma delas:

1. Se a entrada era originalmente uma string (nome e espécie), vemos se ela entrou como uma string vazia ou com um tipo distinto;
2. Se era uma variável do tipo inteiro (preço, estoque e dias para regar), fizemos novamente a checagem de tipo e avaliamos se o valor era válido, como solicitado pelo problema (já que o preço e a constância de regar as flores precisam ser `> 0` e o estoque é necessariamente `>= 0`);
3. Se era uma variável do tipo Luminosidade (que armazena justamente o tipo de luminosidade na qual aquela planta deve ser cuidada), checamos se o elemento nela é compatível com o armazenado no enum;
4. Se é uma variável do tipo booleano (seguro para pets), checamos se a entrada foi recebida como true ou false.

Armazenamos cada uma dessas checagens em uma variável booleana respectiva para cada campo, de forma que o comando condicional if() que leva à mensagem de erro ficasse mais compacto e legível.

Por fim, a criação e armazenamento da planta no dataset ocorrem como no exemplo original, com os códigos de acerto/erro respectivos.

```
    // Cria a planta com os dados recebidos
    const planta = await PlantRepository.createPlant({
      nome: nome,
      especie: especie,
      preco: preco,
      estoque: estoque,
      luminosidade: luminosidade,
      petFriendly: petFriendly,
      regaACadaDias: regaACadaDias
    });

    // Sucesso 201 - conseguimos criar
    return res.status(201).json(planta);

  } catch (error) { // TryCatch - identifica se houve algum erro
    console.error(error);
    // Erro 500: Problema de servidor (nossa parte)
    return res.status(500).json({ message: "Erro ao armazenar dados da planta." });
  }
};

```

Leitura das Plantas (por ID e leitura geral)

As funções de leitura funcionam de forma análoga ao exemplo de usuário, especialmente a função de procura por ID (com a diferença da adição de uma checagem para validar se o identificador existe e, principalmente, se foi transformado no tipo number, já que é recebido como string pela URL).

```
// Encontra uma planta específica pelo seu id
export const findPlantById = async (id: number) => {
    return await prisma.plant.findUnique({
        where: { id: id }
    });
};
```

```
// Função para ler os dados de uma planta específica, de acordo com o id buscado
export const readPlantById = async (req: Request, res: Response) => {
  try {
    // Transforma o id em int, já que na URL é enviado como string
    const id = Number(req.params.id);

    // Checagem para saber se o id foi corrigido corretamente
    let bool_id = Number.isInteger(id) && id > 0;
    if(!bool_id){
      return res.status(500).json({ message: "Problema na conversão de id para inteiro." });
    }

    const plant = await PlantRepository.findPlantById(id);

    // Erro 404 (não encontrado): sem planta com este ID
    if (!plant) {
      return res.status(404).json({ message: "Planta não encontrada." });
    }

    return res.status(200).json(plant);

  } catch (error) {
    console.error(error);
    return res.status(500).json({ message: "Erro ao buscar planta." });
  }
};

```

Entretanto, a função de leitura geral de todas as plantas precisou de um cuidado especial, já que o desafio solicitou a presença de um **filtro opcional** para procurar plantas que atendessem a critérios específicos das variáveis “petFriendly” e “luminosidade”. Para isso, precisamos checar se, no corpo de “query” (onde elas serão recebidas no ThunderClient), os valores de cada uma delas são válidos e não vazios - caso contrário, são definidos como “undefined”, indicando que devem ser ignorados no processo de pesquisa.

```
// Mudança na estrutura do FindAllPlants: agora os filtros opcionais estão presentes para retornar de acordo com o que solicitado
export const findAllPlants = async (filters: { luminosidade?: Luminosidade; petFriendly?: boolean }) => {
    return await prisma.plant.findMany({
        where: {
            luminosidade: filters.luminosidade,
            petFriendly: filters.petFriendly,
        },
        orderBy: { id : 'asc'}
    });
};
```

Aplicamos uma busca com filtros para as variáveis solicitadas.

```
// Lê todas as plantas. Filtros podem ser aplicados dependendo do solicitado (observar PlantRepository)
export const readAllPlants = async (req: Request, res: Response) => {
  try {
    // Pega os filtros e faz a checagem: se não foi chamado, é undefined; se for, adota o valor certo (luminosidade ou booleano)
    const { luminosidade, petFriendly } = req.query;

    // Checa se PetFriendly é undefined ou não; se for, retorna undefined e é ignorado no "where". Se não for, isPetFriendly
    // adota o valor da comparação petFriendly === "true" (já que chega como texto no query) - se sim, isPetFriendly é true (mesmo
    // valor da variável); caso petFriendly === "false", a comparação retorna falso, que é o valor booleano correto
    const isPetFriendly = petFriendly === undefined? undefined : petFriendly === "true";

    // Checa de luminosidade é undefined; se for, retorna undefined e é ignorado no "where". Se não for, luminosidade recebe o
    // tipo do enum Luminosidade, que é o correto.
    let isLuminosidade: Luminosidade | undefined = undefined;

    if(luminosidade !== undefined){ // Se não for undefined, checa se a entrada corresponde a uma entrada válida no Enum
      const isvalidLum = Object.values(Luminosidade).includes(luminosidade as Luminosidade);

      if(!isvalidLum){ // Se não for uma entrada válida dentro do enum, retorna erro
        return res.status(400).json({ message: "Valor de luminosidade inválido." });
      }

      isLuminosidade = luminosidade as Luminosidade; // Muda o valor da variável para o valor do filtro correto
    }

    const plants = await PlantRepository.findAllPlants({
      luminosidade: isLuminosidade,
      petFriendly: isPetFriendly,
    }
      ); // Retorna todas as plantas guardadas que cumpram os requisitos dos filtros; undefineds são ignorados
```

Também checamos se as entradas são válidas - se “luminosidade” possui um valor contido no enum (caso contrário retorna um erro 400 de invalidez do dado recebido) e se “petFriendly” é igual a “true” (caso contrário retorna “false” e pesquisa todas as plantas com esse valor). Caso um dos campos não tenha sido preenchido na URL, é atribuído o valor “undefined”, que faz com que o filtro desconsidere-os.

Atualizações e Deleções de Plantas

Em comparação ao caso original de usuário, a função de deletar não teve alterações além da checagem inicial de validez do identificador da planta buscada.deletePlant deleta por completo a planta de acordo com seu id.

```
    // Atualiza os dados de uma planta específica
    export const updatePlant = async (id: number, data: Prisma.PlantUpdateInput) => {
        return await prisma.plant.update({
            where: { id: id },
            data: data,
        });
    };

    // Deleta uma planta escolhida
    export const deletePlant = async (id: number) => {
        return await prisma.plant.delete({
            where: { id: id }
        });
    };
```

```
// Deleta uma planta existente com base no ID dela
export const deletePlant = async (req: Request, res: Response) => {
  try {
    const id = Number(req.params.id);

     // Checagem para saber se o id foi corrigido corretamente
    let bool_id = Number.isInteger(id) && id > 0;
    if(!bool_id){
      return res.status(500).json({ message: "Problema na conversão de id para inteiro." });
    }

    // Garantia de que a planta que será deletada existe
    const plant = await PlantRepository.findPlantById(id);

    if (!plant) {
      return res.status(404).json({ message: "Planta não encontrada." });
    }

    await PlantRepository.deletePlant(id);

    // Sucesso 204 - Conseguimos deletar!
    return res.status(204).send();

  } catch (error) {
    console.error(error);
    return res.status(500).json({ message: "Erro ao deletar planta." });
  }
};
```

updatePlant, contudo, precisou ser atualizada para checar se os valores recebidos estão de acordo com os tipos originais de cada variável, de forma que ela não admita valores errados para variáveis originalmente corretas após a troca. Para isso, fizemos uma checagem individual para todos os campos, e atualizamos a planta recebida ao final de cada checagem.



```
const dataToUpdate: any = {};

if (nome !== undefined) {
  if (typeof nome !== "string" || nome.trim() === "") {
    return res.status(400).json({ message: "Nome inválido." });
  }
  dataToUpdate.nome = nome;
}

if (especie !== undefined) {
  if (typeof especie !== "string" || especie.trim() === "") {
    return res.status(400).json({ message: "Espécie inválida." });
  }
  dataToUpdate.especie = especie;
}

if (preco !== undefined) {
  if (!Number.isInteger(preco) || preco <= 0) {
    return res.status(400).json({ message: "Preço inválido." });
  }
  dataToUpdate.preco = preco;
}

if (estoque !== undefined) {
  if (!Number.isInteger(estoque) || estoque < 0) {
    return res.status(400).json({ message: "Estoque inválido." });
  }
  dataToUpdate.estoque = estoque;
}

if (regaACadaDias !== undefined) {
  if (!Number.isInteger(regaACadaDias) || regaACadaDias < 1) {
    return res.status(400).json({ message: "Número de dias para rega inválido." });
  }
  dataToUpdate.regaACadaDias = regaACadaDias;
}

if (luminosidade !== undefined) {
  if (!Object.values(Luminosidade).includes(luminosidade)) {
    return res.status(400).json({ message: "Luminosidade inválida." });
  }
  dataToUpdate.luminosidade = luminosidade;
}

if (petFriendly !== undefined) {
  if (typeof petFriendly !== "boolean") {
    return res.status(400).json({ message: "O campo petFriendly deve ser booleano." });
  }
  dataToUpdate.petFriendly = petFriendly;
}
```

Vendas e Total Acumulado no Estoque

Por fim, implementamos as funções “sellPlant” e “totalPlant” que retornam, respectivamente, a planta que foi atualizada com o valor no estoque decrementado e o total de vasos contidos na loja, além do preço total que a loja possui em estoque (somatório de preço * nº no estoque para cada planta).

Para a função sellPlant, fizemos a checagem do id e recebemos um objeto que armazena a variável “quantidade”, que será justamente o valor a ser decrementado no estoque daquela planta específica. Em seguida, checamos se ele corresponde a um valor positivo (afinal, uma quantidade `< 0` não representa uma venda, e sim uma adição ao estoque) e se ele não é maior do que o estoque atual (para evitar que o estoque fique negativo, o que é

inválido), para logo depois realizar a operação de decrement dentro do arquivo de repositório.

```
// Função para realizar as vendas
export const sellPlant = async (id: number, qtd_venda: number) => {
    return await prisma.plant.update({
    where: { id: id },
    data: {
        estoque: { // Decrementa o valor de estoque de acordo com o valor recebido
            decrement: qtd_venda ,
            }
        }
    })
}
```

```
// Função para Decrementar a qtd de plantas no estoque
export const sellPlant = async (req: Request, res: Response) => {
  try{
    const id = Number(req.params.id);
     // Checagem para saber se o id foi corrigido corretamente
    let bool_id = Number.isInteger(id) && id > 0;
    if(!bool_id){
      return res.status(500).json({ message: "Problema na conversão de id para inteiro." });
    }

    const plant = await PlantRepository.findPlantById(id); // Checa se a planta existe
    if(!plant){
      return res.status(404).json({ message: "Planta não encontrada." });
    }

    const qtd_vendas = Number(req.body.quantidade); // Pega o nº que será decrementado do estoque
    // Checagem da correção de id
    let bool_vendas = Number.isInteger(qtd_vendas) && qtd_vendas > 0; // Venda é sempre > 0 (se for 0, não vendeu nada)
    if(!bool_vendas){
      return res.status(500).json({ message: "Problema na conversão do nº de plantas vendidas." });
    }

    if(plant.estoque < qtd_vendas){ // Checa se o estoque é maior do que o valor que será decrementado. Não pode estoque negativo
      return res.status(400).json({ message: "Quantidade no estoque insuficiente." });
    }

    const updatedPlant = await PlantRepository.sellPlant(id, qtd_vendas); // Atualiza o valor da planta com o novo estoque
    return res.status(200).json(updatedPlant);

  } catch (error){
      console.error(error);
      return res.status(500).json({ message: "Erro ao efetuar venda." });
    }
```

Em seguida, na função totalPlant, usamos duas funções diferentes no repositório: a função “totalSales” — que retorna o valor máximo de receita que a loja pode obter caso venda todas as suas plantas — e a função “totalStock” — que retorna o número total de vasos ainda presentes na loja. Nesta última, fizemos uso do operador “??” no return para que, caso a loja esteja vazia (o que resultaria em `total_vases._sum.estoque` ser null graças à propriedade do método aggregate), o valor retornado seja 0, que é o correto.

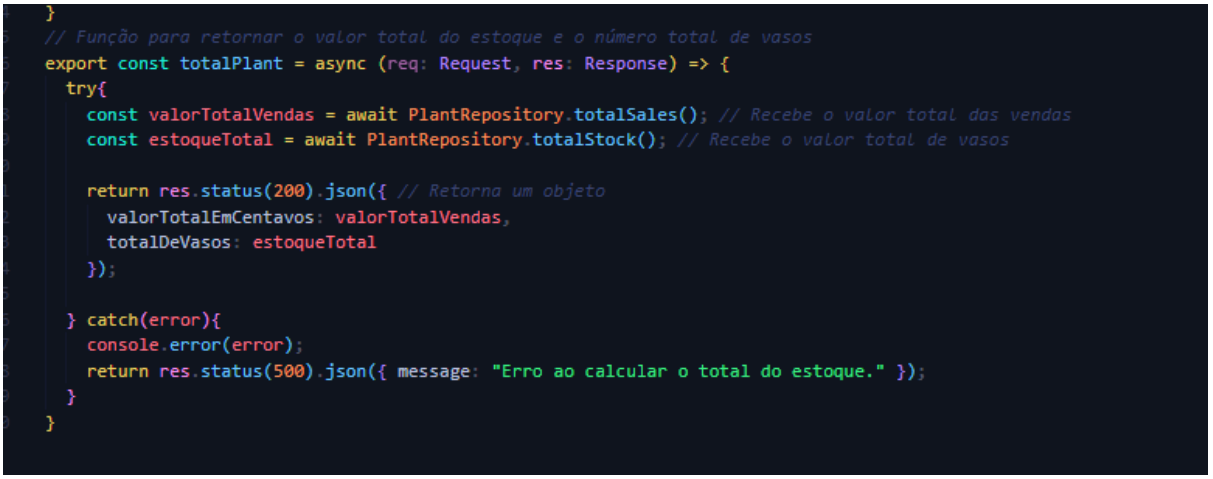
```
// Retorna o total de vendas preço * estoques do dataset
export const totalSales = async () => {
    const plants = await prisma.plant.findMany();

    return plants.reduce((acc, plant) => {return acc + (plant.preco * plant.estoque)}, 0); // Multiplica
}

// Retorna o total de estoque ocupado
export const totalStock = async() => {
     // Soma todos os elementos da coluna de "estoque", retornando o total como um objeto
    const total_vases = await prisma.plant.aggregate({
        _sum: {
            estoque: true,
        },
    });

    return total_vases._sum.estoque ?? 0; // Pega o valor do objeto e transforma null em 0 caso o original esteja vazio
}
```

Por fim, no controlador, fazemos uso dessas duas funções e as unimos em um único objeto de retorno, como solicitado pelo desafio, já que os dados já estão no modelo correto.



```
}
// Função para retornar o valor total do estoque e o número total de vasos
export const totalPlant = async (req: Request, res: Response) => {
  try{
    const valorTotalVendas = await PlantRepository.totalSales(); // Recebe o valor total das vendas
    const estoqueTotal = await PlantRepository.totalStock(); // Recebe o valor total de vasos

    return res.status(200).json({ // Retorna um objeto
      valorTotalEmCentavos: valorTotalVendas,
      totalDeVasos: estoqueTotal
    });

  } catch(error){
    console.error(error);
    return res.status(500).json({ message: "Erro ao calcular o total do estoque." });
  }
}
```

Principais Dificuldades e Uso de IA:

Inicialmente, a dificuldade foi a instalação das ferramentas do Docker e a configuração do ambiente no VS Code. Tive dificuldades para entender o passo a passo fornecido no GitHub original e no documento do desafio, mas com a ajuda do Gemini consegui configurar corretamente e iniciar o projeto.

Na etapa do desafio em si, passei por problemas na aplicação dos filtros opcionais de **luminosidade** e **petFriendly** na função “findAllPlants” do PlantRepository.ts, em que aprendi a usar a função “filters { }” e como transformar os filtros em undefined caso não sejam chamados na URL (ou sejam chamados incorretamente).

Além disso, encontrei problemas de rota e inconsistência na saída dos valores quando implementei as funções “totalSales” e “totalStock”, de forma que, graças à IA e à ajuda de um dos monitores, pude perceber onde se encontravam meus erros, e aprendi a usar o reduce e o aggregate por causa dessa dificuldade.

Na função “updatePlant”, tive dificuldades na forma em que a checagem das variáveis que iriam ser atualizadas deveria ser feita: para isso, pedi para o Gemini gerar um código que solucionasse o problema, e validei cada um dos casos extremos para checar se o código produzido estava de acordo com o que o problema pede.

Por fim, na função “sellPlant” do controlador, tive dificuldade em saber como receber o parâmetro de quantidade, de forma que o Gemini gerou um código base, que adaptei ao problema, e ainda corrigiu problemas na minha rota.

O Claude também foi utilizado na correção do texto final deste documento.

Todas as conversas com o Gemini foram checadas e testadas nos test cases oferecidos pelo desafio, e todos resultaram em sucesso.